

Deep learning Cardiac Cine MRI Segmentation

沈茗祥 叶柯成 黎亚轩 李岳桐 戚昱坤

1 problem (a)

1.1 split the data

```
train_folder = ImageFolder(begin = 0, end = 1030 , mode = 'train')
valid_folder = ImageFolder(begin = 1030, end = 1287 , mode = 'valid')
test_folder = ImageFolder(begin = 1287, end = 1798 , mode = 'test')
```

1.2 key network parameters

- the number of channels in the first stage is 4
- the number of downsampling stages in the encoder is 4

```
class UNet(nn.Module):
    def __init__(self, n_channels, n_classes, C_base=64):
        super(UNet, self).__init__()
        self.n_channels = n_channels
        self.n_classes = n_classes
        self.in_conv = DoubleConv(n_channels, C_base)
        self.down1 = Down(C_base, 2 * C_base)
        self.down2 = Down(2 * C_base, 4 * C_base)
        self.down3 = Down(4 * C_base, 8 * C_base)
        self.down4 = Down(8 * C_base, 16 * C_base)
        self.up1 = Up(16 * C_base, 8 * C_base)
        self.up2 = Up(8 * C_base, 4 * C_base)
        self.up3 = Up(4 * C_base, 2 * C_base)
        self.up4 = Up(2 * C_base, C_base)
        self.out_projection = nn.Conv2d(C_base, n_classes, kernel_size=1)

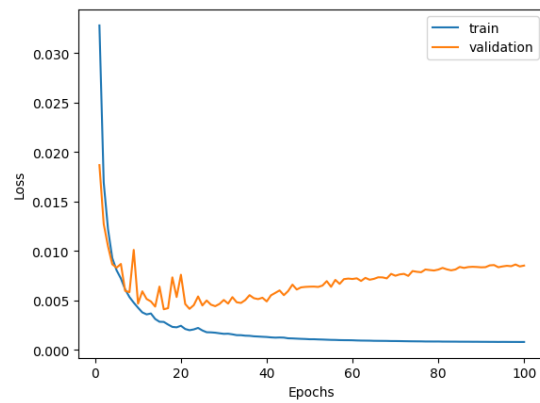
    def forward(self, x):
        x1 = self.in_conv(x)
```

```

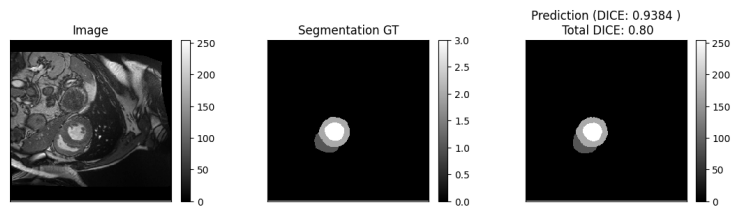
x2 = self.down1(x1)
x3 = self.down2(x2)
x4 = self.down3(x3)
x5 = self.down4(x4)
x = self.up1(x5, x4)
x = self.up2(x, x3)
x = self.up3(x, x2)
x = self.up4(x, x1)
pred = self.out_projection(x)
return pred
net = UNet(n_channels=1, n_classes=4, C_base=32)

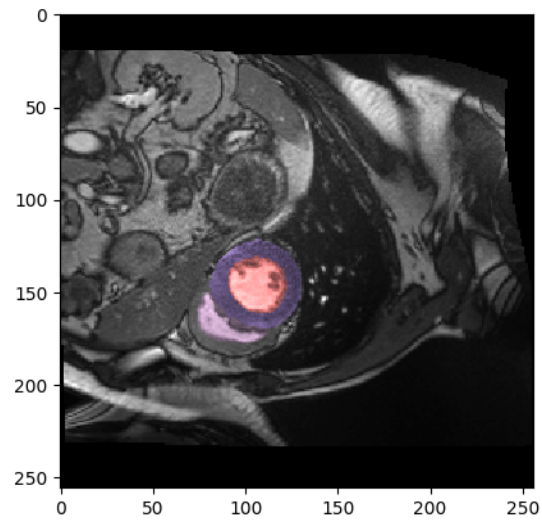
```

1.3 train loss and valid loss



1.4 example segmentation results

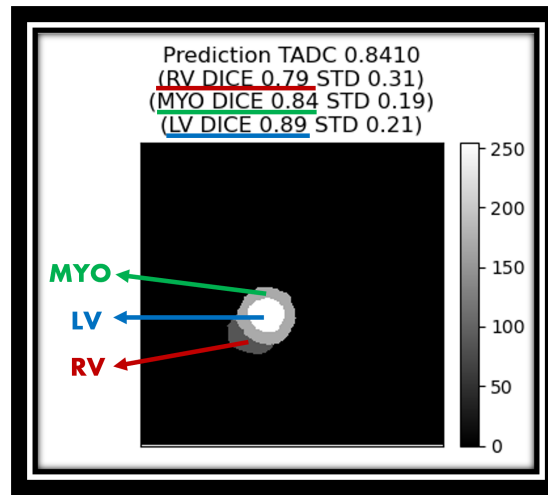




1.5 Dice coefficients : mean and standard deviation of the Dice metrics

name	mean	std
RV	0.64	0.38
MYO	0.73	0.26
LV	0.81	0.30

1.6 comments on the segmentation performance of RV, LV and MYO



- LV has the best performance, followed by MYO and RV has the worst dice loss.
- Possible explanation is because the center part has more significant feature, which is easy to learn, and we found many other study has the similar problem. It can be further researched in future.

2 problem (b)

2.1 comparison with UNet(with and without skip connection)

Main difference lies in up class(delete concatenate).

```
# unet
class Up(nn.Module):
    def __init__(self, in_channels, out_channels):
        super().__init__()
        self.up = nn.ConvTranspose2d(in_channels, in_channels // 2,
                                      kernel_size=2, stride=2)
        self.conv = DoubleConv(in_channels, out_channels)

    def forward(self, x1, x2):
        x1 = self.up(x1)
```

```

H1, W1 = x1.shape[2:]
H2, W2 = x2.shape[2:]
x1 = F.pad(x1, [
    (W2-W1) // 2, # left
    (W2-W1) // 2, # right
    (H2-H1) // 2, # top
    (H2-H1) // 2, # bottom
])
x = torch.cat([x2, x1], dim=1)
out = self.conv(x)
return out

# unet without skip connection
class Up2(nn.Module):
    def __init__(self, in_channels, out_channels):
        super().__init__()
        self.up = nn.ConvTranspose2d(in_channels, in_channels // 2,
                                      kernel_size=2, stride=2)
        self.conv = DoubleConv(in_channels // 2, out_channels)

    def forward(self, x1, x2=None):
        x1 = self.up(x1)
        out = self.conv(x1)
        # without cat
        return out

```

2.1.1 Forward Pass

Original UNet: Each Up layer takes two inputs: the feature map from the corresponding Down layer (skip connection) and the output of the previous Up layer.

UNet without Skip Connections: Each Up layer takes only one input: the output of the previous Up layer. The skip connections are removed.

2.1.2 Up Layer Implementation

- In the original UNet, the Up layer uses concatenation of feature maps.
- In the UNet without skip connections, the Up layer does not concatenate feature maps.

2.1.3 Comparison with result

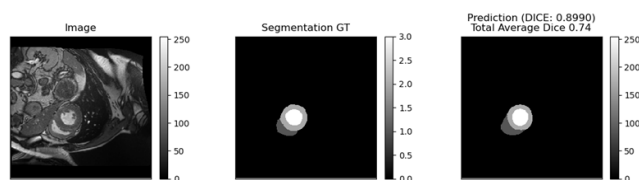


图 1: *withoutskip*, $epoch = 100$

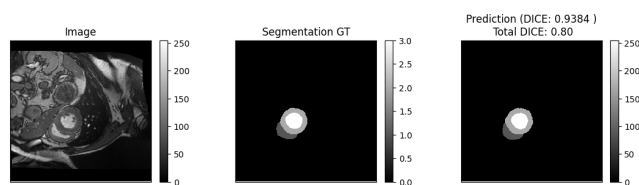


图 2: *Unet*, $epoch = 100$

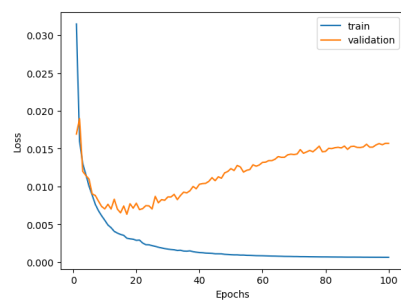


图 3: *withoutskip*, $epoch = 100$

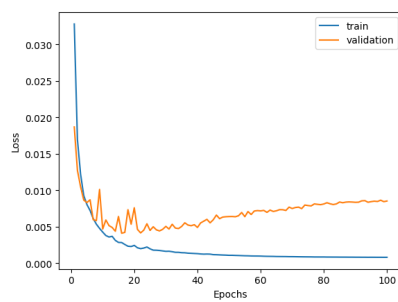


图 4: *Unet*, $epoch = 100$

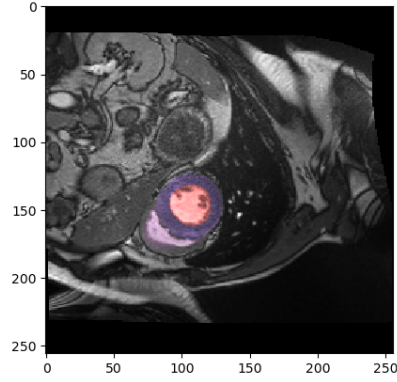


图 5: *withoutskip*, $epoch = 100$

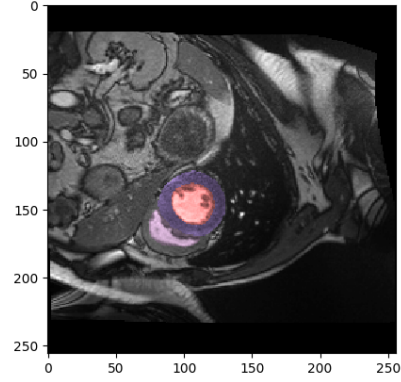


图 6: *Unet*, $epoch = 100$

2.2 explanation

1. U-Net algorithm uses concat to fuse the feature map at the corresponding position in the process of encoding and decoding, so that the decoder can obtain more high-resolution information when upsampling, and then more perfect the detailed information in the original image to improve the segmentation accuracy. U-Net with skip connection structure can make the network fuse the feature map of the corresponding position of the encoder on the channel during the up-sampling process of each level. Through the fusion of low-level features and high-level features, the network can retain more high-resolution detail information contained in high-level feature maps, thus improving the image segmentation accuracy. Cross-layer feature reconstruction enhances the transmission of feature information between layers, and further utilizes the rich details in the high-level convolutional feature layer, thus maximizing the utilization rate of feature information in each layer of the network.

3 problem (c)

3.1 comparison with UNet(with and without data augmentation)

apply augmentation such as rotation, crop, flip with probabilities.

Performance improved from 0.781 to 0.789, which has a slight effect.

```
RotationDegree = random.randint(0, 3)
RotationDegree = self.RotationDegree[RotationDegree]
if (RotationDegree == 90) or (RotationDegree == 270):
    aspect_ratio = 1 / aspect_ratio
Transform.append(T.RandomRotation((RotationDegree, RotationDegree)))
RotationRange = random.randint(-10, 10)
Transform.append(T.RandomRotation((RotationRange, RotationRange)))
CropRange = random.randint(250, 270)
Transform.append(T.CenterCrop((int(CropRange * aspect_ratio), CropRange)
    )))

Transform = T.Compose(Transform)
image = Transform(image)
seg_gt = Transform(seg_gt)
ShiftRange_left = random.randint(0, 20)
ShiftRange_upper = random.randint(0, 20)
ShiftRange_right = image.size[0] - random.randint(0, 20)
ShiftRange_lower = image.size[1] - random.randint(0, 20)
image = image.crop(box=(ShiftRange_left, ShiftRange_upper,
    ShiftRange_right, ShiftRange_lower
    ))
seg_gt = seg_gt.crop(box=(ShiftRange_left, ShiftRange_upper,
    ShiftRange_right, ShiftRange_lower
    ))

if random.random() < 0.5:
    image = F.hflip(image)
    seg_gt = F.hflip(seg_gt)

if random.random() < 0.5:
    image = F.vflip(image)
    seg_gt = F.vflip(seg_gt)
```

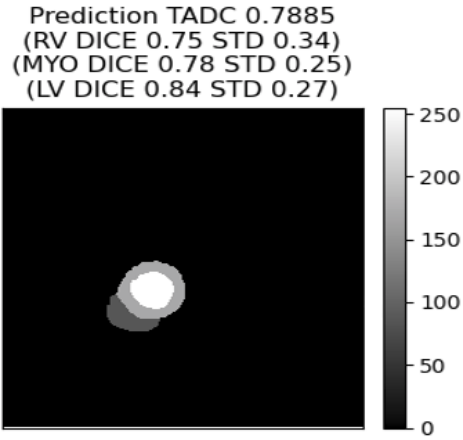



图 7: Data augmentation

4 problem (d)

4.1 our soft dice implement

```
class SoftDiceLoss:
    def __init__(self, epsilon=1e-6, ignore_index=0):
        self.epsilon = epsilon
        self.ignore_index = ignore_index

    def __call__(self, predictions, targets):
        return self.forward(predictions, targets)

    def forward(self, predictions, targets):
        predictions = F1.softmax(predictions, dim=1)
        num_classes = predictions.shape[1]
        one_hot_targets = F1.one_hot(targets, num_classes=num_classes).
            permute(0, 3, 1, 2).float()

        dice_loss = 0.0

        valid_classes = 0
        for class_idx in range(num_classes):
            if class_idx == self.ignore_index:
                continue

            pred_class = predictions[:, class_idx, :, :]
            target_class = one_hot_targets[:, class_idx, :, :]
```

```

intersection = torch.sum(pred_class * target_class)
union = torch.sum(pred_class) + torch.sum(target_class)

dice_coeff = (2 * intersection + self.epsilon) / (union + self.
                                                    epsilon)

dice_loss += 1 - dice_coeff
valid_classes += 1

if valid_classes > 0:
    dice_loss /= valid_classes

return dice_loss

```

4.2 comparison with cross entropy loss

The result is better than using crossentropy loss. The possible analysis is written in the last part of this report.

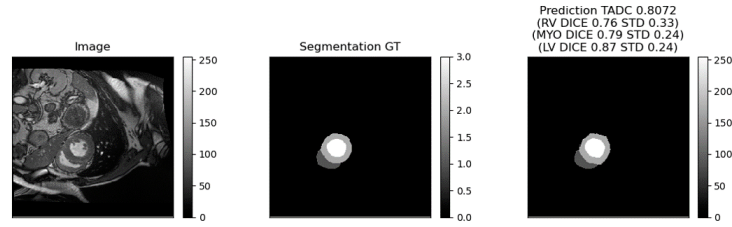


图 8: dice loss

5 problem (e)

5.1 UNet ++ structure

We use the nested UNet which is called UNet++ in [this paper](#) [1].

```

class VGGBlock(nn.Module):
    def __init__(self, in_channels, middle_channels, out_channels):
        super().__init__()
        self.relu = nn.ReLU(inplace=True)
        self.conv1 = nn.Conv2d(in_channels, middle_channels, 3, padding=1)
        self.bn1 = nn.BatchNorm2d(middle_channels)
        self.conv2 = nn.Conv2d(middle_channels, out_channels, 3, padding=1)
        self.bn2 = nn.BatchNorm2d(out_channels)

```

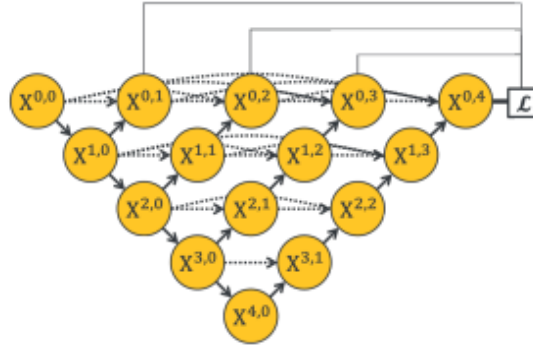


图 9: Nested U-Net in Original Paper

```
def forward(self, x):
    out = self.conv1(x)
    out = self.bn1(out)
    out = self.relu(out)

    out = self.conv2(out)
    out = self.bn2(out)
    out = self.relu(out)

    return out

class NestedUNet(nn.Module):
    def __init__(self, num_classes, input_channels=1, deep_supervision=False,
                  **kwargs):
        super().__init__()

        nb_filter = [32, 64, 128, 256, 512]

        self.deep_supervision = deep_supervision

        self.pool = nn.MaxPool2d(2, 2)
        self.up = nn.Upsample(scale_factor=2, mode='bilinear', align_corners=True)

        self.conv0_0 = VGGBlock(input_channels, nb_filter[0], nb_filter[0])
        self.conv1_0 = VGGBlock(nb_filter[0], nb_filter[1], nb_filter[1])
        self.conv2_0 = VGGBlock(nb_filter[1], nb_filter[2], nb_filter[2])
        self.conv3_0 = VGGBlock(nb_filter[2], nb_filter[3], nb_filter[3])
        self.conv4_0 = VGGBlock(nb_filter[3], nb_filter[4], nb_filter[4])
```

```

self.conv0_1 = VGGBlock(nb_filter[0]+nb_filter[1], nb_filter[0],
                        nb_filter[0])
self.conv1_1 = VGGBlock(nb_filter[1]+nb_filter[2], nb_filter[1],
                        nb_filter[1])
self.conv2_1 = VGGBlock(nb_filter[2]+nb_filter[3], nb_filter[2],
                        nb_filter[2])
self.conv3_1 = VGGBlock(nb_filter[3]+nb_filter[4], nb_filter[3],
                        nb_filter[3])

self.conv0_2 = VGGBlock(nb_filter[0]*2+nb_filter[1], nb_filter[0],
                        nb_filter[0])
self.conv1_2 = VGGBlock(nb_filter[1]*2+nb_filter[2], nb_filter[1],
                        nb_filter[1])
self.conv2_2 = VGGBlock(nb_filter[2]*2+nb_filter[3], nb_filter[2],
                        nb_filter[2])

self.conv0_3 = VGGBlock(nb_filter[0]*3+nb_filter[1], nb_filter[0],
                        nb_filter[0])
self.conv1_3 = VGGBlock(nb_filter[1]*3+nb_filter[2], nb_filter[1],
                        nb_filter[1])

self.conv0_4 = VGGBlock(nb_filter[0]*4+nb_filter[1], nb_filter[0],
                        nb_filter[0])

if self.deep_supervision:
    self.final1 = nn.Conv2d(nb_filter[0], num_classes, kernel_size=1
                             )
    self.final2 = nn.Conv2d(nb_filter[0], num_classes, kernel_size=1
                             )
    self.final3 = nn.Conv2d(nb_filter[0], num_classes, kernel_size=1
                             )
    self.final4 = nn.Conv2d(nb_filter[0], num_classes, kernel_size=1
                             )
else:
    self.final = nn.Conv2d(nb_filter[0], num_classes, kernel_size=1)

def forward(self, input):
    x0_0 = self.conv0_0(input)
    x1_0 = self.conv1_0(self.pool(x0_0))
    x0_1 = self.conv0_1(torch.cat([x0_0, self.up(x1_0)], 1))

    x2_0 = self.conv2_0(self.pool(x1_0))
    x1_1 = self.conv1_1(torch.cat([x1_0, self.up(x2_0)], 1))
    x0_2 = self.conv0_2(torch.cat([x0_0, x0_1, self.up(x1_1)], 1))

```

```

x3_0 = self.conv3_0(self.pool(x2_0))
x2_1 = self.conv2_1(torch.cat([x2_0, self.up(x3_0)], 1))
x1_2 = self.conv1_2(torch.cat([x1_0, x1_1, self.up(x2_1)], 1))
x0_3 = self.conv0_3(torch.cat([x0_0, x0_1, x0_2, self.up(x1_2)], 1))
x4_0 = self.conv4_0(self.pool(x3_0))
x3_1 = self.conv3_1(torch.cat([x3_0, self.up(x4_0)], 1))
x2_2 = self.conv2_2(torch.cat([x2_0, x2_1, self.up(x3_1)], 1))
x1_3 = self.conv1_3(torch.cat([x1_0, x1_1, x1_2, self.up(x2_2)], 1))
x0_4 = self.conv0_4(torch.cat([x0_0, x0_1, x0_2, x0_3, self.up(x1_3)
                                ], 1))

output = self.final(x0_4)
return output

```

5.2 comparison with UNet

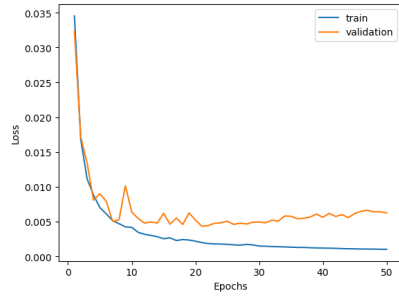


图 10: Unet,epoch=50

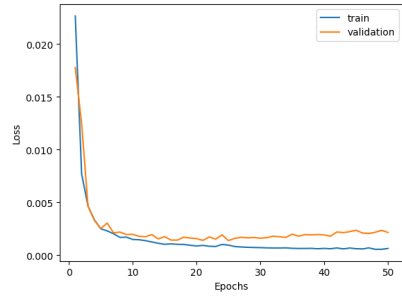


图 11: Unet++,epoch=50

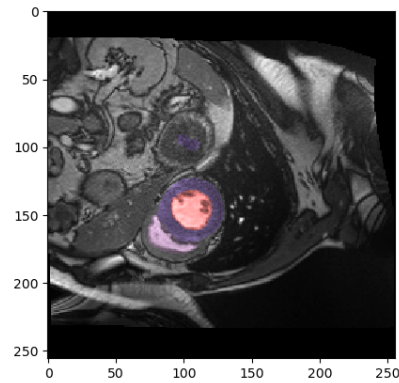


图 12: Unet,epoch=50

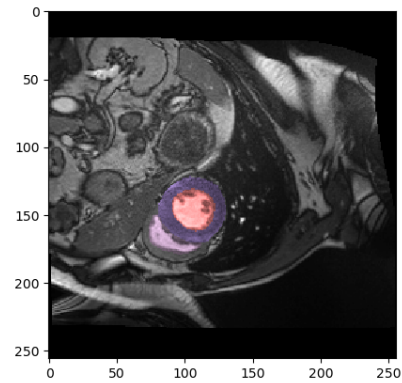


图 13: Unet++,epoch=50

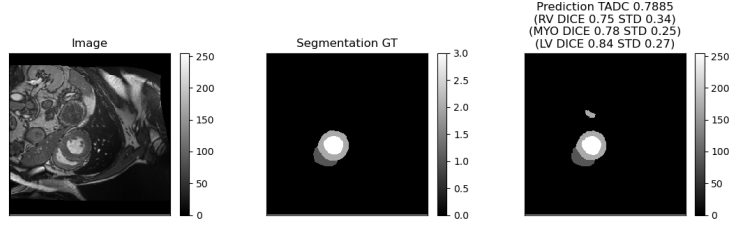


图 14: Unet final result

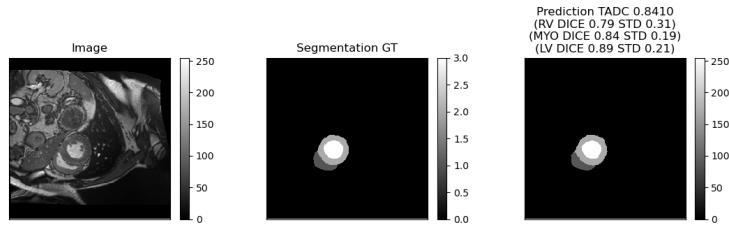


图 15: Unet++ final result

6 conclusion

model	dice loss
unet	0.78
unet without skip connection	0.74
unet++	0.84

U-Net has demonstrated remarkable performance in medical image segmentation, effectively delineating the desired regions with clarity. A significant portion of U-Net’s capabilities stems from its skip connections, and experiments have shown a conspicuous decline in performance when these connections are removed. This degradation is attributed to the lack of global information integration. In response, U-Net++, which incorporates ideas from DenseNet, excels in capturing global, local, and detailed features, and empirical evidence confirms that this improvement is not merely due to an increase in the number of network parameters but rather a consequence of the enhanced network architecture.

Regarding the loss function, Dice loss outperforms cross-entropy loss in

this context. We posit that this superiority arises because the task at hand is not binary classification but rather multi-class segmentation. Therefore, Dice loss is more suitable for this scenario, exhibiting a smoother response compared to cross-entropy.

Lastly, data augmentation also plays a role, contributing to a modest enhancement in network performance by introducing diversity in the training dataset and reducing overfitting.

model comparisons under [epoch = 50]

参考文献

- [1] Zongwei Zhou, Md Mahfuzur Rahman Siddiquee, Nima Tajbakhsh, and Jianming Liang. Unet++: A nested u-net architecture for medical image segmentation. In Danail Stoyanov, Zeike Taylor, Gustavo Carneiro, Tanveer Syeda-Mahmood, Anne Martel, Lena Maier-Hein, João Manuel R.S. Tavares, Andrew Bradley, João Paulo Papa, Vasileios Belagiannis, Jacinto C. Nascimento, Zhi Lu, Sailesh Conjeti, Mehdi Moradi, Hayit Greenspan, and Anant Madabhushi, editors, *Deep Learning in Medical Image Analysis and Multimodal Learning for Clinical Decision Support*, pages 3–11, Cham, 2018. Springer International Publishing.